TON DUC THANG UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

MIDTERM PROJECT REPORT

Data Mining and Knowledge Discovery

Personalized Game Recommendation for Users with Sparse Interaction Histories

Playnext

Lecturer: Hoàng Anh

Group 6 | Class 23K50201

523K0006 – Nguyễn Bá Hùng

523K0014 – Nguyễn Bảo Long

Ho Chi Minh City, September 2026

Abstract

Personalized game recommendation becomes difficult when a user supplies only a few positive interactions. This midterm project investigates that setting through a reproducible data-mining pipeline and a working application named Playnext. The study combines historical Steam reviews from the UCSD V1 collection with Steam metadata from the V2 collection, joining records by application identifier. A deterministic sample contains 20,000 users and 37,391 cleaned positive user–game pairs. Users are partitioned into training, validation, and development-test sets in a 70/15/15 ratio. The training catalog contains 2,235 games, and the binary interaction matrix is approximately 99.916% sparse.

Three baselines are implemented: Most Popular, content-based recommendation with tag/genre TF-IDF and cosine similarity, and sparse item-based collaborative filtering. The main evaluation provides nested support sets of one, three, and five games to the same held-out users while retaining a fixed query set. After catalog filtering, the common validation and test cohorts contain 125 and 123 users. A separate short-observed-history cohort avoids conflating simulated information scarcity with genuinely short records in the source. Recall@10 and NDCG@10 are reported alongside catalog coverage, intra-list diversity, and training popularity.

Most Popular obtains the strongest ranking results in this development experiment. At five support games, its Recall@10 and NDCG@10 are 0.224661 and 0.161285, compared with 0.064905 and 0.042373 for Content and 0.122358 and 0.084229 for ItemCF. Personalized baselines expose a wider catalog, illustrating a measurable trade-off rather than an overall improvement. A local inference benchmark and an independent audit support reproducibility, but neither establishes online effectiveness. The final part describes the website that loads verified artifacts and explains recommendations using actual model signals. This submission freezes the M0-M3 baseline scope. BPR, hybrids, semantic embeddings and multi-seed extensions are reserved for the separate final-term version and are not claimed here.

Keywords: game recommendation; sparse user history; TF-IDF; item-based collaborative filtering; leakage prevention; reproducible evaluation.

Table of Contents

Abbreviations iv
1 Introduction 1
1.1 Problem context 1
1.2 Research questions and project scope 1
1.3 Operational meaning of sparse history 2
2 Foundations and Related Work 2
2.1 Recommendation as knowledge discovery 2
2.2 Content representations and similarity 2
2.3 Collaborative item neighborhoods 3
2.4 Ranking evaluation and popularity bias 3
3 Data Acquisition and Exploratory Analysis 4
3.1 Source provenance and schema 4
3.2 Cleaning and positive-label construction 4
3.3 Sampling and temporal limitations 5
3.4 Distribution of user histories 6
3.5 Item popularity and content coverage 7
4 Data-Mining Pipeline and Baseline Algorithms 8
4.1 End-to-end pipeline and artifact boundaries 8
4.2 Notation and training matrix 8
4.3 Most Popular 9
4.4 Content features and smoothed TF-IDF 10
4.5 Building the user profile and ranking 10
4.6 Sparse ItemCF and neighborhood selection 11
4.7 Fallback, filtering, and deterministic outputs 12
5 Experimental Protocol and Evaluation Metrics 12
5.1 User-disjoint evaluation 12
5.2 Nested support and a fixed query 13
5.3 Catalog filtering and cohort attrition 13
5.4 Short histories and zero support 14
5.5 Relevance metrics and a worked example 15
5.6 Coverage, diversity, and tuning discipline 15
6 Results and Interpretation 16
6.1 Main ranking results 16

6.2 Catalog exposure and diversity 18
6.3 Short-observed-history results 19
6.4 Failure frequency and a concrete case 20
6.5 Target popularity and alternative explanations 20
7 Efficiency, Reproducibility, and Research Limitations 21
7.1 Local inference benchmark 21
7.2 Training time and memory scope 22
7.3 Verification and reproducible evidence 23
7.4 Threats to validity 23
7.5 Research extensions after the midterm 24
8 Website Integration and User Interface 24
8.1 From offline artifacts to online recommendations 24
8.2 Explanations and interface behavior 24
8.3 Demonstration evidence and remaining product work 25
9 Conclusion 26
References 27
Appendix A Reproduction and Evidence Map 28
Appendix B Team Contributions and Responsibilities 29
Shared responsibilities 29
Completion status 29

List of Figures

Figure 1. Distribution of cleaned positive histories in the 20,000-user sample. Source: frozen M1 data_summary.json. 6

Figure 2. Training-item popularity by descending rank on logarithmic axes. Source: frozen popularity array. 7

Figure 3. Ten most frequent tags in the training catalog. Frequencies count games, not users or review events. 7

Figure 4. Research pipeline. Validation selection is completed before development-test scores are computed. 8

Figure 5. Illustrative nested-support protocol. Letters are hypothetical items, not dataset records. The query remains F–H at all three support levels. 13

Figure 6. Common-cohort retention after training-catalog checks. Empty-query exclusions are two validation users and one test user. 14

Figure 7. ItemCF validation objective for the tested neighborhood grid. The objective averages common-cohort NDCG@10 over k=1,3,5. 16

Figure 8. Common-cohort Recall@10 with a fixed query for each user. Source: frozen test metrics. 17

Figure 9. Common-cohort NDCG@10. Content is not monotonic as support grows. 18

Figure 10. Catalog coverage of the three baselines. Coverage is a fraction of the same 2,235-item catalog. 18

Figure 11. Number of common test users with no query hit at support 5. No fallback occurred in these three common-cohort model runs. 20

Figure 12. Local inference p95 by support level. The measurement excludes HTTP, SQL, authentication, explanation construction, and serialization. 21

Figure 13. Frozen midterm recommendation interface from the recorded browser verification. The screenshot illustrates product integration, not an experimental ranking case. 25

Abbreviations

CF — Collaborative Filtering

CSR — Compressed Sparse Row

IDF — Inverse Document Frequency

ILD — Intra-List Diversity

NDCG — Normalized Discounted Cumulative Gain

TF-IDF — Term Frequency–Inverse Document Frequency

BPR-MF — Bayesian Personalized Ranking with Matrix Factorization

API — Application Programming Interface

1 Introduction

1.1 Problem context

A game catalog can contain many titles that differ in genre, mechanics, player interaction, and style. A recommendation interface must reduce that space to a small set of useful alternatives. The difficulty is especially visible at the beginning of a user's interaction with a service: the system may know one favorite game, three recently declared preferences, or nothing beyond a few chosen tags. A method that works with a long interaction history cannot automatically be assumed to work under these conditions.

Playnext studies this problem at the scale of a university midterm project. The main technical objective is to connect a defensible offline experiment to an operational recommendation flow. The research component therefore precedes the interface: the source must be inspected, positive interactions must be defined, training and evaluation boundaries must be established, and results must be reproducible. A polished website alone would not demonstrate that the recommender extracts useful patterns from data.

The central task is Top-10 ranking. Given a set of support games supplied by a held-out user, a model orders eligible games that are not already included in that support. The offline evaluator compares the recommendation list with withheld positive games. This is different from classifying game genres, predicting a rating on a five-star scale, or estimating a purchase probability. No calibrated probability model is implemented, and ranking scores must not be interpreted as percentages of personal suitability.

1.2 Research questions and project scope

The first research question asks how Most Popular, a content model, and an item-based collaborative model compare when one, three, or five support interactions are available. The second asks whether metadata and collaborative evidence can eventually be combined more effectively through a hybrid model. The third concerns the trade-off between relevance, diversity, and exposure of less-popular games. The fourth examines whether training and inference costs are practical on the development machine.

The midterm scope answers the first question with three implemented baselines and begins the fourth with a measured local benchmark. Coverage and diversity provide initial evidence relevant to the third question, but no diversity-oriented reranking experiment has been conducted. Fixed and history-adaptive hybrid methods are part of the subsequent research milestone. Their potential benefits are hypotheses rather than findings of this report.

The project does not claim a novel recommendation algorithm. Its present contribution is an integrated, auditable implementation: a real-data survey, a train-only feature pipeline, a controlled limited-support evaluation, persisted artifacts, measured baseline results, and a website that serves model-derived recommendations. This contribution is appropriate for establishing an academically sound midterm foundation before attempting more complex models.

1.3 Operational meaning of sparse history

Three notions must be separated. A zero-support user supplies no known game interaction to the model. A limited-support user supplies one, three, or five games, regardless of how many games that person has played outside the system. A short-observed-history user has only a small number of positive records in the available review source. None of these definitions proves that a person is inexperienced or inactive in real life.

The main experiment uses users with at least six positive records and reveals only a small prefix of their support pool. This provides a controlled comparison of information availability. The supplementary cohort uses users with two to five observed positives. These populations answer different questions and are reported separately. A user with a single positive cannot provide both one independent support interaction and one withheld target, so that user is not eligible for the one-support ranking test.

Zero-support behavior is evaluated as fallback functionality. The application may display popular games or use independently declared tags, but the offline study does not manufacture onboarding tags from hidden query games. Doing so would reveal the intended answers through metadata and make the resulting evaluation misleading.

2 Foundations and Related Work

2.1 Recommendation as knowledge discovery

The knowledge-discovery process in this project converts semi-structured reviews into representations that support ranking. The stages are source validation, cleaning, integration, interaction construction, exploratory analysis, feature extraction, model fitting, and evaluation. Each stage changes what can legitimately be inferred from the data. For example, joining only records with metadata improves feature availability but also excludes part of the original review population.

The system extracts two kinds of structure. Content features describe properties shared by games, such as tags and genres. Collaborative features describe which games are positively reviewed by overlapping training users. Popularity is a third, simpler signal: the frequency of positive training users for each item. These signals can disagree. Two games may share tags while attracting different audiences, or they may be co-reviewed without having highly similar metadata.

The observed signal is a positive recommendation boolean. It is explicit as a review-level expression of approval, but the ranking matrix retains only positive events and has no complete preference labels for unobserved games. Consequently, an empty matrix cell means unknown rather than verified dislike. This distinction matters for interpreting both collaborative similarity and future negative sampling.

2.2 Content representations and similarity

Content-based recommendation represents each item as a vector and constructs a user profile from known items. In Playnext, the vocabulary contains namespaced tag and genre tokens. A

multiword tag is one feature, and a genre token is distinct from a tag token with the same text. This avoids accidental token fragmentation and retains the origin of each feature.

TF-IDF downweights features that appear in many catalog items and gives relatively more weight to rarer features. The implementation follows the smoothed inverse-document-frequency convention of scikit-learn, while its precise tokenization and train-catalog restriction are project decisions [2]. Each item vector is normalized to unit Euclidean length. Cosine similarity then measures directional agreement rather than raw feature count.

This representation is intentionally simple and interpretable. It cannot directly model narrative quality, visual style, difficulty, price sensitivity, hardware requirements, or a user's preference for different genres in different contexts. It also treats the profile as a single direction. Combining a strategy game and a shooter can therefore produce a mixed vector that does not correspond to a single coherent preference category.

2.3 Collaborative item neighborhoods

Item-based collaborative filtering estimates relationships between items from interaction patterns. Sarwar and colleagues describe item–item similarity and aggregation as an alternative to user-neighborhood recommendation [3]. Playnext adopts the item-based principle, but uses its own binary positive matrix, sparse cosine calculation, row-wise neighborhood truncation, and unnormalized support-score sum. Results from the cited paper are not claimed as results of this implementation.

Let two game columns contain binary entries indicating positive training users. Their dot product counts shared users. Dividing by the product of their vector lengths yields cosine similarity. If two games share no observed positive users, their similarity is zero, even if their genres appear closely related. Thus, collaborative similarity reflects the available interaction sample rather than an intrinsic relationship between games.

Keeping only the strongest N neighbors per item limits artifact size and inference work. It also changes the similarity graph: even if the full cosine matrix is symmetric, independent row truncation can retain one direction while dropping the other. This is not a numerical error. It is a consequence of the neighborhood policy and must be considered when explaining an aggregated score.

2.4 Ranking evaluation and popularity bias

Recall measures the fraction of withheld targets recovered, whereas discounted cumulative gain rewards relevant items that appear early. The normalized gain framework provides a foundation for position-sensitive ranking evaluation [4]. Playnext uses binary relevance, a cutoff of ten, and an ideal ranking based on the valid query size. This explicit convention is necessary because implementations using different gain or discount definitions need not produce identical values.

Accuracy alone does not describe which part of a catalog is exposed. Popularity bias research examines how frequently observed items can dominate recommendations and how that concentration may differ across users [5]. This motivates reporting coverage and recommendation popularity alongside ranking quality. It does not imply that every

recommendation of a popular game is undesirable, nor that low-popularity recommendations are inherently useful.

In this project, diversity is measured through distances in the same tag/genre representation used by Content. The resulting ILD is a representation-dependent statistic. It is not a direct measurement of perceived diversity, satisfaction, novelty, or fairness. A high-diversity list with no relevant games can still fail the user's immediate goal.

3 Data Acquisition and Exploratory Analysis

3.1 Source provenance and schema

The input consists of the UCSD Steam V1 review file and the Steam V2 metadata file linked from the UCSD recommender-systems dataset collection [1]. The downloaded review archive contains 6,935,922 bytes; the metadata archive contains 2,740,516 bytes. The local manifests record source URLs, access information, and SHA-256 checksums. These hashes identify the exact files used, rather than relying on a potentially changing filename.

The review file contains one Python dictionary literal per line, with a user identifier and a list of reviews. It is not a flat JSON review table. The parser uses ast.literal_eval and reports invalid input with its line number. Reviews expose fields including item_id, recommend, posted, last_edited, helpful, funny, and review text. Metadata includes application identifiers, title or app_name, tags, genres, and other partially populated attributes.

The pipeline joins review item_id with metadata id. Names are not used as join keys because titles can change, contain punctuation variants, or refer to different editions. Combining V1 reviews and V2 metadata also means that the records do not form a single synchronized snapshot. Missing joins and potentially later metadata descriptions remain threats to interpretation, even when identifier matching is exact.

The report treats the files as historical research inputs. It does not claim that they represent the current Steam catalog or all Steam players. Redistribution terms have not been established as an unrestricted open license, so the source handoff excludes raw reviews and user handles. User identifiers in processed artifacts are hashed with a namespace; this reduces direct exposure but is not presented as a complete anonymization guarantee.

3.2 Cleaning and positive-label construction

The cleaning sequence begins with metadata validation. Records without identifiers are excluded, repeated metadata identifiers retain their first record, and titles fall back to app_name when necessary. Review rows are flattened into user–item events. User and item identifiers and the recommendation boolean are checked before interaction construction. Duplicate user–item pairs are collapsed, and conflicting positive/negative labels would cause that pair to be excluded. No conflicting pair was observed in the actual survey.

Only pairs with recommend exactly equal to True become positive interactions. A truthy string is not accepted as equivalent to a boolean. After deduplication, positive pairs are joined to metadata and missing matches are counted. Raw negative reviews remain in the source archive, but they are not included as positive matrix entries. The pipeline does not reinterpret

ownership, playing time, helpful votes, review sentiment, or website clicks as equivalent labels.

Table 1. Whole-source cleaning survey.

Whole-source survey measure	Observed count
User rows / unique users	25,799 / 25,485
Reviews / reviewed items	59,305 / 3,682
True / False recommendations before deduplication	52,473 / 6,832
Metadata rows / unique metadata IDs	32,135 / 32,132
Excess reviews within repeated user–item pairs	874
Reviews without a metadata match	5,317
Unique positive pairs before metadata join	51,692
Positive pairs excluded by the metadata join	4,200
Positive pairs / items after metadata join	47,492 / 2,869

The difference between row counts and unique entities is important. A duplicated user row is not a new person, and a repeated review is not independent preference evidence. Similarly, the missing-join rate for all reviews and the missing-join rate for positive pairs use different denominators. Reporting both stages makes it possible to reconstruct why the final modeling population is smaller than the raw source.

No user k-core filter is applied. Removing users who have few interactions would simplify some collaborative algorithms but undermine the stated research focus. Users with zero cleaned positives remain in the sampled user population for descriptive accounting, although they cannot contribute targets to the ranking evaluation. This preserves visibility of exclusions instead of silently retaining only easy-to-model users.

3.3 Sampling and temporal limitations

The first feasibility sample contained 5,000 users. After the planned user split and training-catalog restrictions, only 14 common validation users and 23 common test users remained. The sample was therefore expanded to 20,000 users before examining model quality scores. The M0 sample was retained separately, and the expansion followed a recorded decision rather than selecting a sample that produced a favorable ranking outcome.

Sampling is deterministic: unique user identifiers are ordered, the configured seed controls selection, and all cleaned positives belonging to a selected user are retained. The M1 sample contains 37,391 positive pairs and 2,614 games. Splitting users produces 14,000 training users, 3,000 validation users, and 3,000 development-test users. The training catalog is subsequently restricted to 2,235 items with a positive training interaction and joined metadata.

The posted field records review publication rather than game-play time. The full survey parsed 49,186 dates with sufficient year information and counted 10,119 records without a

complete valid date. Missing years are not inferred, and edit time is not substituted for interaction time. The chosen random holdout consequently supports a static historical recommendation study; it cannot establish performance on future behavior in chronological deployment.

3.4 Distribution of user histories

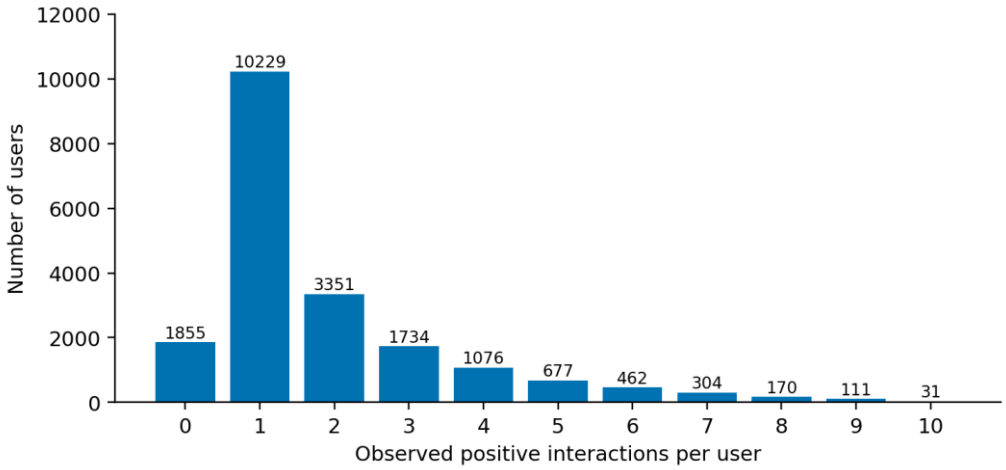


Figure 1. Distribution of cleaned positive histories in the 20,000-user sample. Source: frozen M1 data_summary.json.

The distribution in Figure 1 is dominated by very short records. There are 1,855 users with zero positive pairs and 10,229 with exactly one, corresponding to 9.275% and 51.145% of the sample. Those groups together make up more than half of the user sample, but they cannot directly support the main one/three/five-support experiment with an independent target. The reported ranking cohort is therefore much smaller than the number of sampled users.

The largest observed history has ten positives. This upper bound should not be interpreted as a behavioral law about Steam users. It may reflect the way the source was collected, but the cause has not been verified. The project does not extrapolate to the planned 6–20 and above-20 interaction bands because the available positive histories do not support a meaningful above-20 cohort.

The difference between sample size and evaluation size is a central reporting issue. Saying that a recommender was evaluated on 20,000 users would be incorrect here. That number describes the sampled source population. The main test ranking averages use 123 eligible users after splitting, support construction, and catalog filtering. Both figures are useful, but they answer different questions.

3.5 Item popularity and content coverage

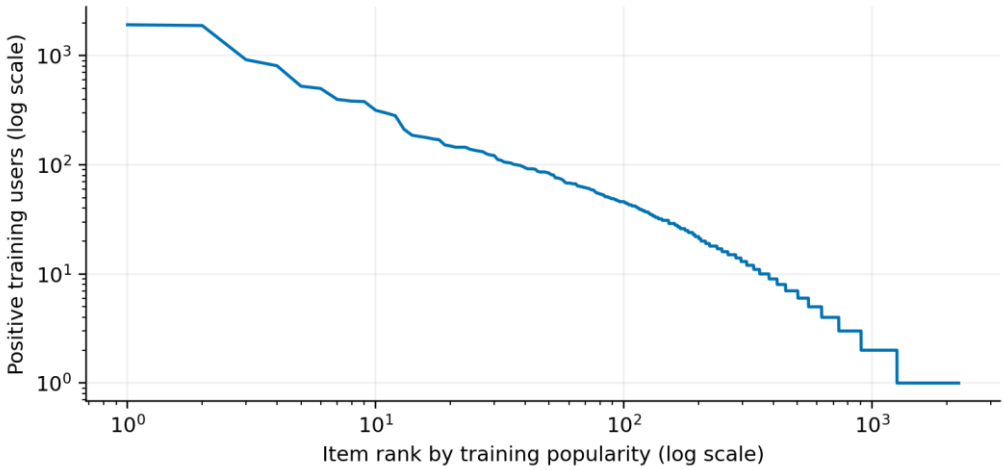


Figure 2. Training-item popularity by descending rank on logarithmic axes. Source: frozen popularity array.

The training interaction matrix contains 26,174 nonzero entries across 14,000 users and 2,235 games. Its density is only about 0.08365%, or approximately 99.91635% sparsity. A sparse CSR representation stores observed pairs efficiently and avoids creating a dense user-by-item array. More importantly, the low density explains why many item pairs may have little or no direct co-occurrence evidence, although it does not by itself prove the cause of any model's test error.

Figure 2 shows that interaction frequency is uneven across training items. A small popular head coexists with a long tail. This makes Most Popular a meaningful baseline rather than a trivial comparison to omit. It also motivates examining whether a method improves exposure of less-observed items at the cost of recovering fewer known query games.

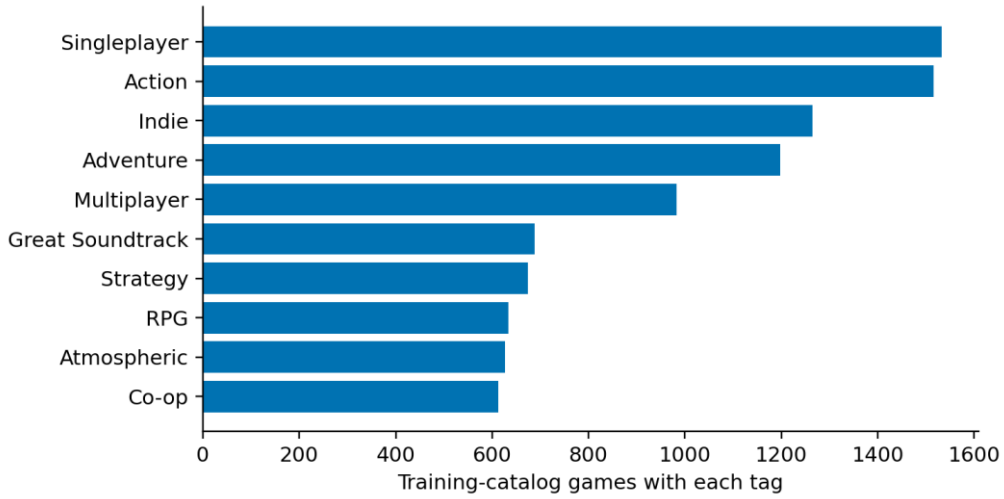


Figure 3. Ten most frequent tags in the training catalog. Frequencies count games, not users or review events.

Content features remain available for every item in the actual training catalog: the saved feature matrix has 349 dimensions and no item with an entirely empty feature vector. This does not imply perfect metadata quality. A few broad labels can appear in many games and provide weak discrimination between individual preferences. The distinction between feature

availability and feature usefulness is important when assessing the relatively low Content ranking scores.

4 Data-Mining Pipeline and Baseline Algorithms

4.1 End-to-end pipeline and artifact boundaries

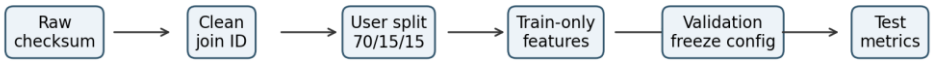


Figure 4. Research pipeline. Validation selection is completed before development-test scores are computed.

The entry point ml.run verifies raw checksums, reconstructs the configured sample, validates processed interactions, and generates the user split. Model fitting uses training histories only. The same held-out case-generation function constructs validation and test cases independently of the model under evaluation. Validation trials determine the ItemCF neighborhood parameter, after which a selection file records the chosen setting and its timestamp before test metrics are computed.

The run directory stores configuration, data summaries, cohort audits, validation trials, the selected setting, aggregate metrics, and a manifest. The artifact directory stores split identifiers, evaluation cases, per-user predictions, and model arrays. Separating human-readable reports from numerical artifacts supports inspection without embedding the entire dataset in the application database. The manifest links the components through hashes of raw files, inputs, code, dependencies, configuration, and outputs.

The present frozen run is 20260915T050327853533Z-seed42. A second run using the same configuration reproduced metrics, cohorts, splits, features, and CF values. This checks deterministic behavior, but two executions with one seed are not a multi-seed study. Reproducibility of a fixed experiment and robustness across random partitions are separate properties.

4.2 Notation and training matrix

Let U_{tr} denote training users, C the common training catalog, and R the binary interaction matrix. Entry R_{ui} equals one when user u has a cleaned positive interaction with game i and zero otherwise. The zero is a storage convention for an unobserved positive; it is not a verified negative preference. For a held-out user u, S_u is supplied support and Q_u is the valid withheld query. A candidate list is drawn from C excluding S_u.

Table 2. Notation used in the baseline models.

Symbol	Meaning in this implementation
C	The 2,235-item training catalog shared by all baselines
R	Binary training user–item CSR matrix
x_i	L2-normalized tag/genre TF-IDF vector for item i
p_u	L2-normalized sum of support item vectors
W_{ji}	Retained item-neighborhood similarity from j to i
S_u and Q_u	Supplied support and disjoint valid query
k and K	Number of support games and ranking cutoff, respectively

Keeping k distinct from K prevents a common ambiguity: the study varies support k in {1,3,5}, while every reported recommendation metric uses K=10. The number of supplied games is not the number of results requested. The catalog restriction is also fixed across models, so Content is not allowed to score extra metadata-only items while collaborative methods are evaluated on a smaller set.

4.3 Most Popular

Most Popular scores an item by the number of training users who positively interacted with it. Because user–item pairs are unique after cleaning, summing a binary column yields the desired count. Duplicate reviews cannot inflate that count. The model needs no held-out user vector and no hyperparameter search in the current experiment.

$$s_{\text{pop}}(i) = \sum_{u \in U_{\text{tr}}} R_{ui} \qquad (1)$$

At inference time, the algorithm copies the popularity vector, excludes supplied support and hidden games, and sorts remaining candidates by descending score. Ties are resolved using the stable lexicographic order of source identifiers. The deterministic tie rule ensures that repeated calls and artifact reloads do not change the list arbitrarily.

The baseline is useful in two ways. It establishes how much quality can be achieved without a personal profile, and it provides a defined fallback for missing support or absent content/collaborative signal. However, a popular result is not inherently personalized. If Recall increases from one to five support items, that increase may arise because additional already-known games are removed from the candidate list, allowing another popular query game into the Top-10.

Its count vector requires one numeric value per catalog item. The current recommend routine still filters and sorts the candidate set for each request. Thus, the local benchmark measures the actual implementation rather than a theoretical constant-time lookup of a universal list. A future implementation might optimize repeated popularity filtering, but no such optimization is required to interpret the measured baseline.

4.4 Content features and smoothed TF-IDF

For each game, the model creates a set of tags and genres, trims whitespace, applies case folding, and prefixes each token with its field name. A tag such as “open world” remains one atomic feature. The same text appearing as both a tag and a genre occupies two dimensions, which preserves source type but can give related concepts multiple contributions. This is an explicit representation choice that future ablations could examine.

Let n be the number of training-catalog items and df(t) the number whose metadata contains token t. The smoothed inverse document frequency and item normalization are:

$$\mathrm{idf}(t) = \log \frac{1+n}{1+\mathrm{df}(t)} + 1 \tag{2}$$

$$\mathbf{x}_i = \frac{\mathbf{v}_i}{\|\mathbf{v}_i\|_2}, \quad v_{it} = \mathrm{tf}_{it}\,\mathrm{idf}(t) \tag{3}$$

Since the token set removes within-item duplicates, tf_{it} is effectively one for present tokens and zero for absent tokens. A repeated tag in the source therefore does not amplify an item's vector. Fitting the vocabulary and IDF only on the training catalog prevents evaluation-only items from changing the feature weights. No review text or global sentiment statistic enters this representation.

As a small illustrative calculation, suppose a token appears in two of three hypothetical training items. Its smoothed IDF is log(4/3)+1, approximately 1.2877. A token appearing in only one has IDF log(4/2)+1, approximately 1.6931. The rarer token receives more weight before item normalization. These numbers illustrate the formula only; they are not measurements from the Steam experiment.

The learned vocabulary and IDF are saved with the item feature matrix. This matters operationally because rebuilding TF-IDF from a changing catalog inside an API request would alter scores and invalidate the connection between an online model version and the offline experiment. The application loads the frozen transform instead.

4.5 Building the user profile and ranking

The content profile aggregates only supplied support vectors and normalizes their sum. Equal support weighting is used; review recency, review length, and hours played do not affect the profile. Because the final vector is normalized, summing and averaging support vectors produce the same direction whenever the aggregate is nonzero.

$$\mathbf{p}_u = \frac{\sum_{j \in S_u} \mathbf{x}_j}{\| \sum_{j \in S_u} \mathbf{x}_j \|_2} \tag{4}$$

$$s_{\text{content}}(u, i) = \mathbf{x}_i^T \mathbf{p}_u \tag{5}$$

The dot product is cosine similarity between unit vectors. A candidate gains score from dimensions that it shares with the profile, weighted by TF-IDF. This creates a natural explanation mechanism: each dimension contributes x_{it} times p_{ut}. Displaying the largest positive contributions gives an evidence-based reason for a recommendation without inventing a textual story about the user's intent.

More support does not mathematically guarantee better ranking. Adding a vector changes the direction of the normalized sum. A newly added game may represent a different interest, causing the profile to move away from a withheld query. It can also add very common features that make many candidates similarly scored. The observed Content NDCG decline from k=1 to k=3 is therefore not automatically an implementation defect.

The approach is inspectable and inexpensive at this catalog size, but it models one aggregate preference direction. It does not retain separate interest clusters or learn which support game should be trusted more. Such extensions would need validation experiments and a clear leakage-safe objective; they are not implicitly present in the current baseline.

4.6 Sparse ItemCF and neighborhood selection

For ItemCF, each item is represented by a column of R. The implementation normalizes these item–user vectors, multiplies them in sparse blocks, removes self-similarity and zero entries, and retains the strongest N similarities per row. Blocks of 128 items bound intermediate processing rather than constructing a full dense similarity matrix.

$$\text{sim}(i, j) = \frac{R_{:i}^T R_{:j}}{\|R_{:i}\|_2 \|R_{:j}\|_2} \tag{6}$$

$$s_{\text{CF}}(u, i) = \sum_{j \in S_u} W_{ji} \tag{7}$$

For example, in a hypothetical binary matrix, an item reviewed positively by four users and another by nine users with two users in common have cosine similarity 2/(sqrt(4) sqrt(9))=1/3. This is an explanatory example, not an extracted Steam item pair. A single shared user can produce a relatively large similarity for two rare items, which is one reason to interpret unregularized cosine carefully.

Playnext sums retained similarities from the support rows. It does not normalize this sum by the number of contributing neighbors, apply rating mean-centering, or add shrinkage. The absence of these extensions makes the baseline easier to reproduce, but also identifies concrete future comparisons. A claimed improvement from adding shrinkage would need to be measured under the same candidates and held-out cases.

The neighborhood grid is N in {20,50,100}. Validation selection maximizes the mean common-cohort NDCG@10 over support levels one, three, and five. If configurations tie, the smaller N is selected. The observed validation objective increases across the tested grid, and N=100 is selected. This only establishes the best value among the tested candidates; it does not establish that 100 is a globally optimal neighborhood size.

4.7 Fallback, filtering, and deterministic outputs

The recommendation interface accepts a model name, support, a Top-K cutoff, and optional hidden games. It does not accept query targets. Unknown support identifiers are ignored when building the profile. If no known support remains, Content and ItemCF fall back to training popularity. They also fall back when the entire score vector has no positive content or collaborative signal.

The fallback decision is made on the score vector before candidate exclusion. A partially zero-scored candidate set is not filled with a second popularity ranking. Zero-score candidates can remain in the returned list according to the stable identifier tie rule. Accordingly, an explanation for such a candidate must report missing similarity evidence rather than describe a strong match. This distinction is useful for both debugging and user-facing honesty.

Support and hidden items are removed consistently before selecting the final Top-K. The offline experiment does not invent dislike or wishlist events from query targets; those signals belong to the application. Tests verify disjointness, unknown-ID handling, finite scores, deterministic behavior, fallback, and artifact reload. An implementation is not considered correct merely because it returns ten plausible game names.

5 Experimental Protocol and Evaluation Metrics

5.1 User-disjoint evaluation

The unit of the primary split is the user, not the interaction. Training users are disjoint from validation and development-test users. This prevents a collaborative model from learning a held-out user's historical representation during fitting and later presenting it as a new-user inference result. A user identifier is used only to make case construction deterministic, not as an input feature that carries their withheld history into scoring.

Splits use a stable input order and a fixed random seed. The training portion is 70%, validation is 15%, and the remainder is test. All three baselines use the same split, catalog, case manifests, and filtering policy. If the dataset or catalog changes in a future milestone, all baselines must be rerun before presenting a direct comparison.

The development-test label is deliberate. These results have already been viewed during project development. They cannot later be relabeled as an untouched final test. Future research must state how the final configuration and evaluation will be locked, and what prior exposure to the existing test permits or limits.

5.2 Nested support and a fixed query

For a common-cohort user with at least six cleaned positive items, a deterministic permutation is generated from the experiment seed and a hash of the user identifier. The first five items form the support pool. All remaining items form the query before catalog filtering. The supplied support at k=1, k=3, and k=5 is a nested prefix of that same pool.

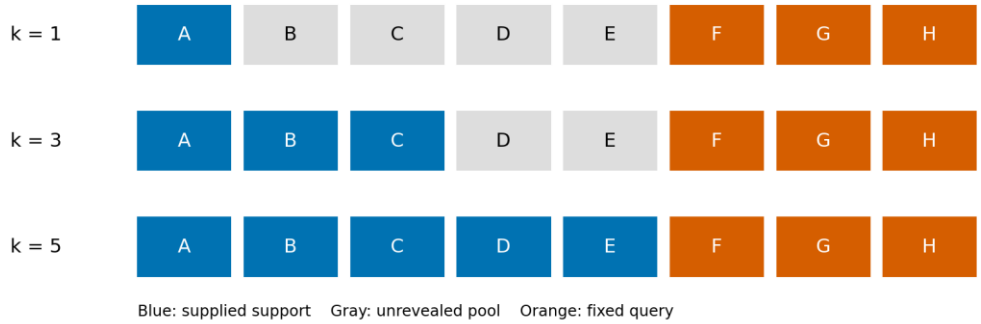


Figure 5. Illustrative nested-support protocol. Letters are hypothetical items, not dataset records. The query remains F–H at all three support levels.

In Figure 5, a user has eight hypothetical items A–H. At k=1 only A is supplied. B–E remain unrevealed pool items; they are not reassigned as query targets. At k=3, A–C are supplied, and at k=5 the full A–E pool is supplied. F–H remain the query throughout. This design ensures that changing support size does not simultaneously replace the evaluation target set.

Only support actually supplied at that level is excluded from candidates. Therefore, unrevealed pool items can appear in a ranking without counting as query hits. This is a specific protocol choice rather than an oversight. It avoids filtering candidates using hidden support information, although it also means that the offline target set does not exhaust every item that the user may like.

5.3 Catalog filtering and cohort attrition

The main catalog contains only items with a positive training interaction and available metadata. Common-cohort admission requires all five support-pool items to lie in that catalog, even when evaluating k=1. After query filtering, at least one valid target must remain. This policy preserves the same user population across all three support levels.

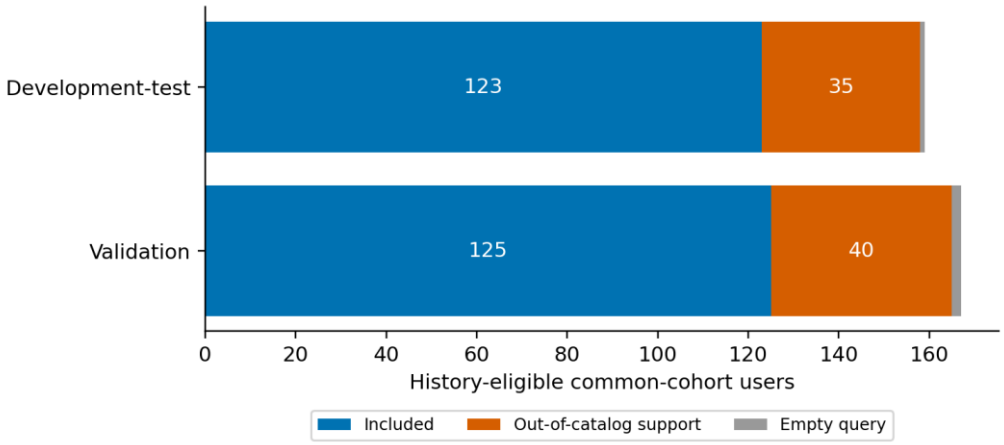


Figure 6. Common-cohort retention after training-catalog checks. Empty-query exclusions are two validation users and one test user.

Of 167 history-eligible validation users, 40 are excluded because at least one support-pool item is outside the catalog and two more because no valid query remains. The retained count is 125. Of 159 eligible test users, 35 fail the support check and one more has an empty valid query, leaving 123. These are eligibility and filtering outcomes, not a sample chosen by model performance.

The common query audit counts 11 out-of-catalog query occurrences among 334 original validation occurrences and ten among 333 test occurrences. Those denominators include all history-eligible users, including users rejected for support. They must not be confused with the 258 valid query occurrences in the retained common test cohort. Reporting the scopes explicitly avoids apparent inconsistencies between cohort and target tables.

For the common cohort, candidate counts are 2,234, 2,232, and 2,230 at support levels one, three, and five. Evaluation uses the complete candidate catalog; there is no sampled-negative shortcut. This increases interpretability because a score is obtained under the same candidate difficulty for each model at a given support level.

5.4 Short histories and zero support

The supplementary short-observed cohort uses users with between k+1 and five positive items. At k=1, eligible users have two to five positives, while k=3 requires four or five. After catalog checks, the test cohorts contain 986 and 212 users respectively. A k=5 cohort does not exist under this definition because five support items plus an independent target would require at least six positives.

The two short-history averages cannot be compared as if they represented the same individuals with additional information. Their membership changes with k. By contrast, the common cohort is designed for within-population comparisons across support levels. Reporting both cohorts makes the project more informative without mixing their interpretations.

Zero support is handled through tested application behavior. Popularity is the default when no preferences are available; independently declared tags can form a content query. However, the historical review source does not supply a separate onboarding-tag response for each test user. The project therefore does not claim offline ranking quality for that tag-based cold-start flow.

5.5 Relevance metrics and a worked example

Recall@10 divides the number of recommended query games by the size of the valid query set. It measures recovery of known targets without considering their order within the Top-10. NDCG@10 adds position sensitivity through a logarithmic discount and normalizes against an ideal list for the same query size. Both metrics are computed per user and then macro-averaged.

$$\text{Recall@10}(u) = \frac{|R_u[:10] \cap Q_u|}{|Q_u|} \tag{8}$$

$$\text{DCG@10}(u) = \sum_{r=1}^{10} \frac{\text{rel}_r}{\log_2(r+1)} \tag{9}$$

$$\text{NDCG@10}(u) = \frac{\text{DCG@10}(u)}{\sum_{r=1}^{\min(10,|Q_u|)} 1/\log_2(r+1)} \tag{10}$$

Consider an illustrative query with three items, two of which appear at ranks one and four. Recall is 2/3. DCG is 1/log2(2)+1/log2(5), approximately 1.430677. IDCG is 1+1/log2(3)+1/log2(4), approximately 2.130930, giving NDCG approximately 0.671386. These are hand-calculation examples, not fabricated experimental rows. Such examples anchor metric unit tests to known answers.

Macro averaging gives each eligible user equal weight even if query sizes differ. Pooling all hits and targets would instead weight users with more targets more heavily and would not reproduce the reported Recall. Empty-query users are counted and excluded before evaluation; empty cohorts yield null metrics rather than zero quality. Duplicate recommendation identifiers are rejected rather than counted repeatedly.

5.6 Coverage, diversity, and tuning discipline

Coverage@10 is the number of distinct recommended catalog items across the cohort divided by the common training-catalog size. A method can increase coverage by returning different obscure items to different users, even if few of those items match their query. Coverage must therefore be read jointly with relevance metrics rather than treated as an alternative definition of accuracy.

ILD averages one minus cosine similarity over pairs of recommended items with nonzero content vectors, then averages over eligible lists. Lists with fewer than two valid vectors do not contribute and their exclusion count is retained. Average recommendation popularity uses training positive-user counts. These metrics describe exposure and metadata distance, not perceived enjoyment or sales impact.

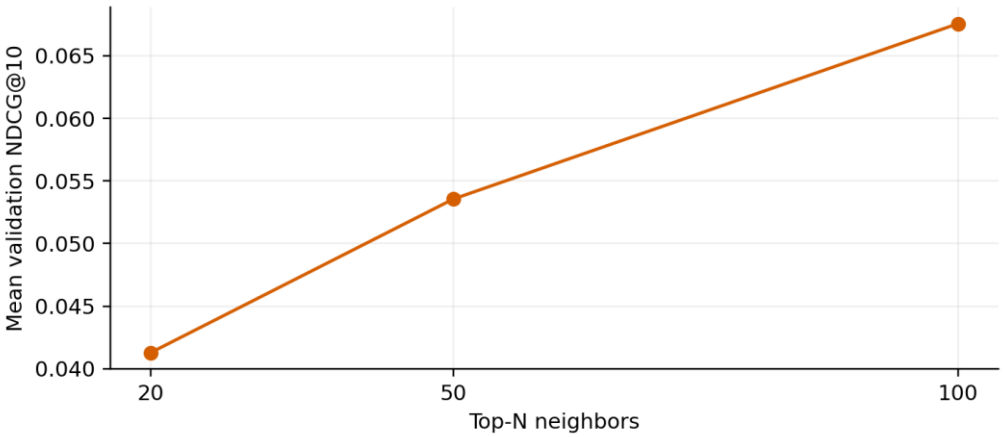


Figure 7. ItemCF validation objective for the tested neighborhood grid. The objective averages common-cohort NDCG@10 over k=1,3,5.

Table 3. ItemCF validation selection.

Neighbors N	Validation mean NDCG@10
20	0.041283
50	0.053553
100	0.067556

The selected N=100 is saved before development-test scoring. Popular and Content have no tuning grid in M1. The trials remain available even when they perform worse, and the test results are not used to choose another seed. This ordering is part of the evidence, not merely a statement of intent in the report.

6 Results and Interpretation

6.1 Main ranking results

The following results use the frozen seed-42 run, the common catalog, and the same 123 development-test users. They summarize the current implementation; they do not establish statistical significance, since repeated-seed estimates and confidence intervals have not been measured. Every model receives exactly the same support for a given case.

Table 4. Common development-test ranking results.

Model	Support	Recall@10	NDCG@10	Coverage@10
Popular	1	0.215176	0.146111	0.004922
Content	1	0.038889	0.034630	0.255481
ItemCF	1	0.036450	0.023343	0.285906
Popular	3	0.220596	0.156797	0.005817
Content	3	0.046341	0.024265	0.219687
ItemCF	3	0.089566	0.054812	0.302013
Popular	5	0.224661	0.161285	0.006711
Content	5	0.064905	0.042373	0.187919
ItemCF	5	0.122358	0.084229	0.283221

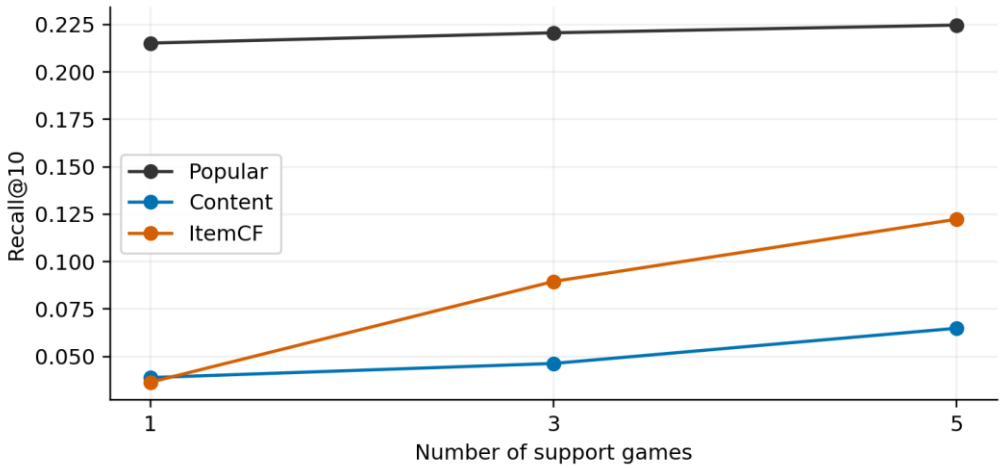


Figure 8. Common-cohort Recall@10 with a fixed query for each user. Source: frozen test metrics.

Most Popular is the strongest of the three methods at every measured support level. At k=5, ItemCF remains 0.102303 below Popular in Recall and 0.077056 below it in NDCG. Content also remains substantially lower. The appropriate conclusion is that the present personalized baselines have not improved held-out ranking quality in this development setting, rather than that personalization is successful because the application returns individually varying lists.

ItemCF improves from Recall 0.036450 at k=1 to 0.122358 at k=5 on the common cohort. That pattern is consistent with additional support supplying more collaborative neighborhood evidence. However, one seed does not establish a general law, and the contribution of candidate filtering changes at the same time. A more complete analysis would use repeated seeds and user-level paired comparisons with the protocol held fixed.

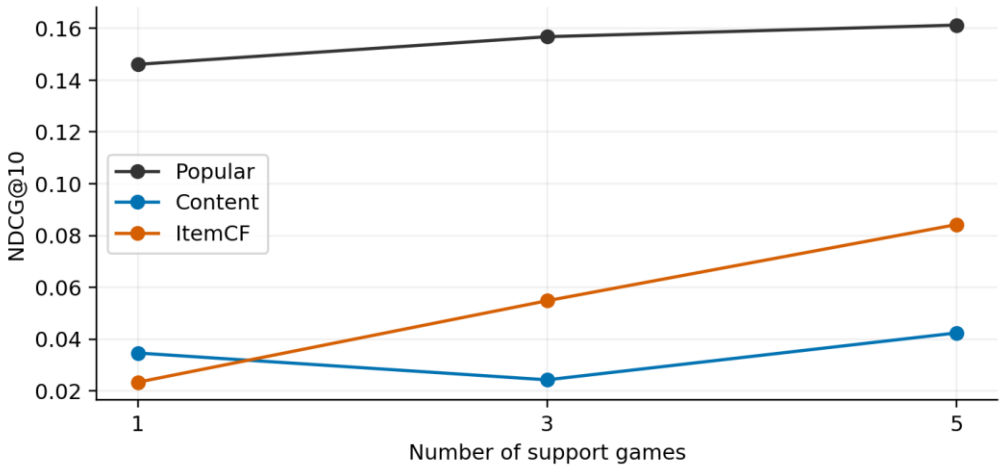


Figure 9. Common-cohort NDCG@10. Content is not monotonic as support grows.

Content NDCG falls from 0.034630 at k=1 to 0.024265 at k=3 before rising to 0.042373 at k=5. This is a useful negative result. Averaging more metadata vectors does not guarantee that relevant games move upward. The report retains this behavior instead of selecting only a favorable support level or replacing the split after seeing the scores.

6.2 Catalog exposure and diversity

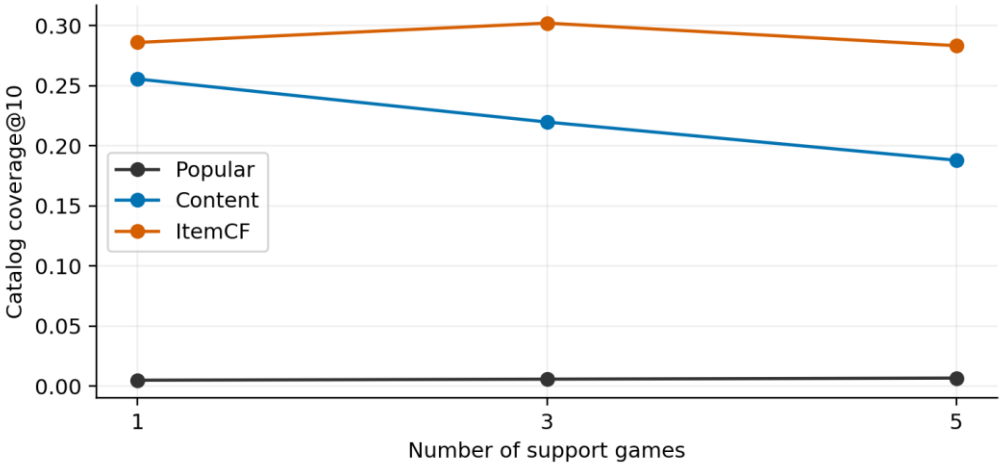


Figure 10. Catalog coverage of the three baselines. Coverage is a fraction of the same 2,235-item catalog.

Popular's k=5 coverage is 0.006711, approximately 0.67% of the catalog. Content reaches about 18.79%, and ItemCF about 28.32%. These differences show that the personalized methods distribute recommendations over a much broader set of games. They do not cancel out the ranking-quality deficit: broad exposure and target recovery are distinct properties.

Table 5. Exposure and diversity at support five.

Model at support 5	ILD	Mean training popularity
Popular	0.737230	719.221
Content	0.538722	36.271
ItemCF	0.865081	240.493

Content has lower intra-list diversity than Popular in this representation, despite its much higher catalog coverage. There is no contradiction. Coverage aggregates across users, while ILD describes differences within one user's list. A content model may return a narrow cluster for each user but a different cluster for different users. ItemCF produces the highest k=5 ILD, although its lists still recover fewer targets than Popular.

The mean popularity values provide a further exposure perspective. Content recommends items with far fewer positive training users on average, whereas Popular concentrates on the head. This is descriptive evidence of different ranking behavior. It does not establish that Content is fairer, more novel to the user, or better for long-term discovery; those outcomes require additional definitions and measurements.

6.3 Short-observed-history results

Table 6. Short-observed-history ranking results.

Model	Support	Users	Recall@10	NDCG@10
Popular	1	986	0.322262	0.212619
Content	1	986	0.045892	0.034621
ItemCF	1	986	0.108942	0.068714
Popular	3	212	0.283019	0.186630
Content	3	212	0.084906	0.044075
ItemCF	3	212	0.143868	0.081467

Popular also leads in the supplementary short-observed cohorts. These results provide evidence beyond the common cohort, but the population definitions remain narrower than “all inactive Steam users.” Only observed positive records determine eligibility, and the same metadata and training-catalog restrictions apply.

The higher Popular Recall at short-history k=1 than at k=3 must not be interpreted as harm caused by adding support. The 986 and 212 users are different populations with different history and query-size distributions. This is exactly why the main experiment keeps a common user cohort and fixed query when studying support length.

6.4 Failure frequency and a concrete case

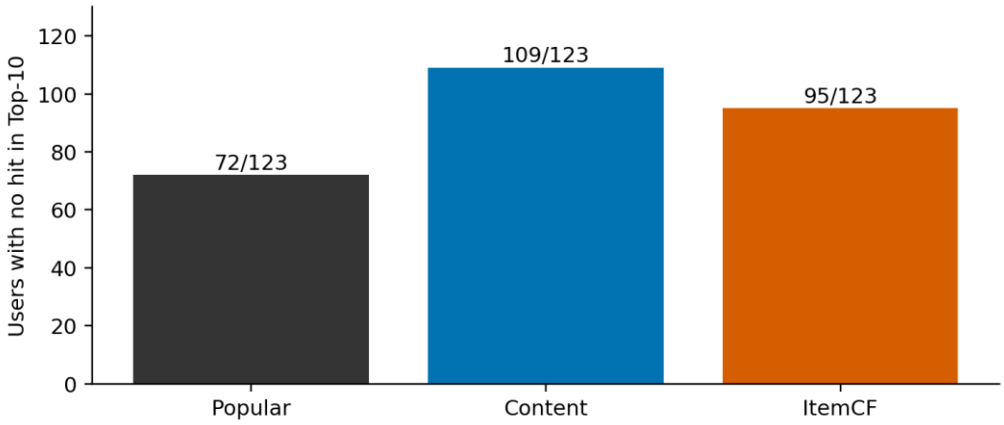


Figure 11. Number of common test users with no query hit at support 5. No fallback occurred in these three common-cohort model runs.

At k=5, 72 of 123 users have no Top-10 hit under Popular, 109 under Content, and 95 under ItemCF. Even the strongest average baseline therefore fails to recover any known query item for many users. An average metric should not hide that distribution. At the same time, a no-hit list is not proof that every recommended game is disliked: the query is an incomplete set of known positives.

One reproducibly selected case illustrates the gap between semantic plausibility and measured target recovery. The selection rule chooses the first internal user key, in sorted order, among common-test k=5 cases where Popular has a hit while Content and ItemCF have none. The report omits the user identifier. This deliberately selected failure case explains behavior and is not a representative sample or evidence about overall prevalence.

The five support games are Surgeon Simulator, Prison Architect, Left 4 Dead 2, Dota 2, and Counter-Strike: Global Offensive. The query contains Terraria, Half-Life 2, and Cities: Skylines. Popular ranks Terraria fifth, obtaining Recall 1/3 and NDCG approximately 0.181542. Content's first five results are Counter-Strike: Source, Left 4 Dead, Insurgency, Natural Selection 2, and Killing Floor, and none of its Top-10 matches the query. ItemCF also has no query hit in this selected case.

The Content results are intuitively connected to part of the support, especially multiplayer and shooter preferences. The query, however, includes different aspects of the user's observed interests. This example shows why a persuasive explanation or visually plausible game list cannot replace ranking evaluation. It also suggests a hypothesis about mixed interests and profile averaging, but it does not prove that hypothesis or justify changing parameters based on the test case.

6.5 Target popularity and alternative explanations

The retained common test cohort contains 258 valid query occurrences, of which 57 belong to the ten most popular training games. This concentration is consistent with a setting in which a popularity baseline can recover a meaningful fraction of targets. It is not sufficient to attribute the entire performance gap to popularity bias: the source, sample, cohort restrictions, and representation all interact.

Several plausible explanations remain untested. Broad tags may fail to distinguish fine-grained interests. Sparse co-occurrence may leave useful item relationships unobserved. Row-wise Top-N truncation may drop weak but relevant links. The positive-review source may emphasize widely reviewed games. Each possibility should be investigated with a controlled ablation rather than asserted from one aggregate score table.

The project therefore separates observations from explanations. Observations include the recorded ranking gaps, coverage differences, no-hit counts, and query popularity concentration. Explanations are hypotheses that guide M4 experiments. Preserving this separation makes negative results useful and prevents an attractive narrative from overstating what the data actually show.

7 Efficiency, Reproducibility, and Research Limitations

7.1 Local inference benchmark

The benchmark runs on an Intel Core i7-11800H with eight physical cores, sixteen logical threads, and approximately 15.79 GiB RAM, using Python 3.11.5 on Windows. Numeric libraries are limited to one thread for the benchmark. Each model runs in a new subprocess that loads the frozen baseline bundle. The workload uses 125 common validation users at each support level, one complete warm-up, and five measured repetitions per level.

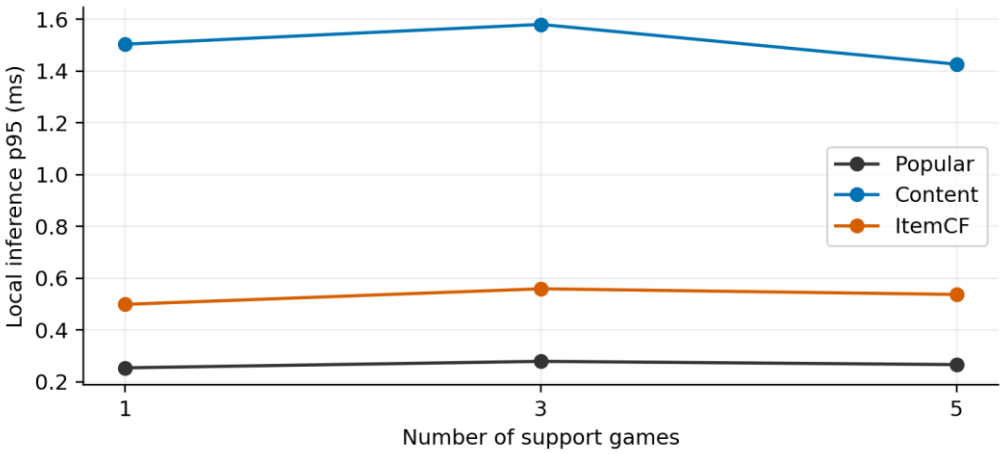


Figure 12. Local inference p95 by support level. The measurement excludes HTTP, SQL, authentication, explanation construction, and serialization.

Table 7. Local inference latency by model and support.

Model	Support	Calls	p50 in ms	p95 in ms
Popular	1	625	0.2236	0.2542
Popular	3	625	0.2395	0.2793
Popular	5	625	0.2352	0.2667
Content	1	625	1.0264	1.5039
Content	3	625	1.0870	1.5803
Content	5	625	1.0523	1.4266
ItemCF	1	625	0.2984	0.4993
ItemCF	3	625	0.3361	0.5595
ItemCF	5	625	0.3385	0.5376

There are 5,625 measured calls in total. The measured function includes scoring, filtering, and sorting, and repeated output is checked for consistency. The table supports the conclusion that these baseline calculations are lightweight at the current catalog size on this machine. It does not show that the complete API has sub-millisecond response time, nor that the system sustains a particular number of concurrent clients.

The benchmark machine is also a normal development workstation, without controlled thermal conditions or background workloads. Five repetitions within a subprocess do not measure variability across independent hardware or operating-system sessions. The results are useful engineering evidence with a stated scope rather than a general performance guarantee.

7.2 Training time and memory scope

The recorded M1 pipeline takes 19.078 seconds including survey reconstruction, fitting, tuning, evaluation, and EDA export, but excluding dataset download. The initial fit is approximately 0.0992 seconds. The distinction is important: end-to-end experiment time is dominated by more than the numerical estimator and should not be reported as the model's online latency.

The M1 process peak working set is approximately 224.05 MiB. In the M3 benchmark, peak subprocess working sets are about 127 MiB and include the Python runtime, libraries, workload, and all three baseline artifacts. They are not isolated per-model peak memory measurements. The smaller numeric-buffer sizes describe selected array storage only and omit Python objects and shared infrastructure.

Model persistence uses NPZ and JSON rather than pickle. Sparse features, CF, and interactions are saved separately; arrays store popularity and IDF; metadata stores the catalog, vocabulary, schema version, and neighborhood setting. Shape, finite-value, schema, and checksum checks reduce the risk of loading an incompatible or damaged artifact while keeping the inference path separate from training.

7.3 Verification and reproducible evidence

An independent M1 audit checks 3,138 evaluation cases, 9,414 predictions, and 36 aggregate metric rows. It validates input and artifact hashes, disjoint splits, train-only interaction statistics, support/query boundaries, common candidates, and recomputed ranking/coverage values. The audit is rerun for this isolated submission without retraining or modifying the frozen quality results; the accompanying submission verification records the outcome.

The repository also contains tests for safe parsing, cleaning edge cases, manually calculated metrics, nested support, catalog filtering, fallback, hidden/support exclusion, and artifact round trips. The isolated submission reruns unit and database/API tests, frontend compilation and the frozen artifact audit. The dated verification file records the exact commands and outcomes, separately from the historical development evidence.

The source handoff contains small experiment reports and reproducibility instructions, but excludes raw data, generated model artifacts, environment secrets, database state, and installed dependencies. A recipient must download the pinned source files and regenerate artifacts or obtain a separately authorized artifact copy. A fresh Python environment has been checked on the same machine; an independent teammate-machine installation remains unverified.

7.4 Threats to validity

Source selection limits external validity. Review authors are not a random sample of Steam users, and positive recommendations do not describe the full set of games a user enjoys. The cross-version metadata join removes some games and may attach information from a different historical moment. The resulting study is static and historical, not a forecast of future preferences.

Cohort selection also matters. The common cohort requires six positives and five support-pool items within the training catalog. It excludes many sampled users and can favor better-observed games. The short-history cohort reduces one aspect of this restriction, but still requires valid support and query items. Neither setting establishes effectiveness for all real-world new users.

Statistical uncertainty remains unmeasured. The project has one experimental seed, no confidence intervals, and no significance test. The report consequently avoids claiming that the observed ordering must hold under another sample or partition. It also avoids comparing these values directly with results from another paper whose dataset, target definition, candidates, or split may differ.

Finally, offline relevance is incomplete. Unobserved candidates can be genuinely interesting, and low query overlap does not prove universal user dissatisfaction. Conversely, an attractive list with a convincing explanation does not prove quality. A voluntary usability study could assess clarity and ease of use, but it would not substitute for an adequately designed recommendation-quality evaluation.

7.5 Research extensions after the midterm

BPR-MF is a planned extension rather than an implemented result. Bayesian Personalized Ranking learns preferences through pairwise comparisons between observed positive and unobserved items [6]. For held-out users, the intended fold-in procedure fixes item factors and estimates a new user factor using support only. Query interactions must not influence that optimization, and a pre-trained embedding for the held-out user would violate the new-user setup.

Fixed hybrid scoring will combine consistently scaled content and collaborative scores with a validation-selected weight. An adaptive alternative may make that weight depend on the number of support games. Reranking can then trade relevance against content similarity within a candidate shortlist. All components require validation selection, ablations, and comparisons under the same catalog and case manifests.

The next research milestone also plans seeds 42, 43, and 44, with every run retained and reported rather than selecting the best seed. Data expansion must be preceded by a feasibility check and versioned separately. BPR quality, hybrid gains, reranking effects, multi-seed uncertainty, tag-onboarding quality, isolated model memory, and API load performance are currently not measured.

8 Website Integration and User Interface

8.1 From offline artifacts to online recommendations

The website connects React/TypeScript/Vite to a FastAPI backend and PostgreSQL database. The backend loads a verified model artifact bundle and checks its relationship to the model registry and experiment reports. Training does not occur inside a recommendation request. Historical dataset users are not converted into login accounts, and the transactional database does not need to contain every raw review.

The current online path uses Content when game support is available, Popular when no support or usable preference signal exists, and a train-vocabulary tag profile when a zero-game-support user independently selects known tags. ItemCF is evaluated offline but is not the default online method. The application does not automatically switch to the offline winning model or claim that its content flow has improved measured quality.

User-declared favorite games and positive feedback update effective support. Dislike removes a game from support and hides it from subsequent recommendations. Wishlist records saved items separately and does not silently create positive training evidence; clicks are logged but are not treated as approval. These distinctions keep the product's interaction semantics consistent with the report's caution about labels.

8.2 Explanations and interface behavior

For content recommendations, the explanation reports high-contributing shared features and actual support references. The contribution of a feature follows the product of the candidate and profile coordinates. A zero-score candidate receives a missing-signal explanation rather

than an invented similarity claim. Popularity-based output is identified as such. Ranking scores are not displayed as calibrated probabilities.

The user flow includes account registration, catalog search, selecting one/three/five support games, requesting suggestions, inspecting a game, saving it, hiding it, and returning after a reload. This frozen interface uses Vietnamese labels with catalog search, preference selection and persistent account actions. Later bilingual and visual redesigns belong to the separate final-term application.

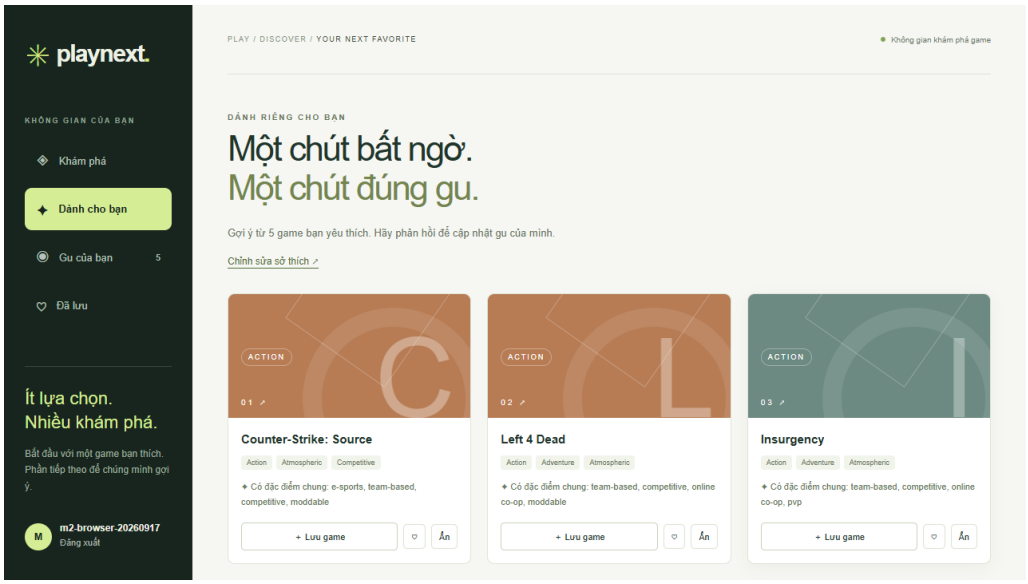


Figure 13. Frozen midterm recommendation interface from the recorded browser verification. The screenshot illustrates product integration, not an experimental ranking case.

The submission does not include the later Steam artwork or animated-background upgrade. The research dashboard is restricted to administrators and reads stored metrics from the real frozen experiment; illustrative or generated values are not substituted for measurements.

8.3 Demonstration evidence and remaining product work

Recorded browser checks exercise registration, zero/one/three/five-support states, saving and hiding games, reload, and login persistence. Database/API tests check authentication, access boundaries, mutation semantics, migrations, and idempotent seeding. These checks support the claim that the interface is connected to persistent application behavior rather than being a static mockup.

The product still has engineering work beyond the midterm scope. Automated browser regression in CI, production load testing, login rate limiting, backup/restore, model rollback, and independent-machine setup have not all been accepted. Local inference timing must not be used as evidence that these operational requirements are complete.

The website's role in this report is to demonstrate how a frozen, evaluated recommender can be served and explained. It does not change the research conclusion. The strongest measured baseline remains Popular, and future algorithm upgrades must be evaluated before their benefits are advertised through the interface.

9 Conclusion

The midterm project establishes a complete data-mining workflow for limited-history game recommendation. Real source files are validated and cleaned, training features and collaborative statistics are isolated from held-out users, nested support sets permit a controlled comparison, and metrics can be traced to saved per-user predictions. The pipeline and artifact boundary also support a functioning full-stack application.

The principal empirical finding is that Most Popular outperforms the implemented Content and ItemCF baselines in the current seed-42 development experiment, while personalized methods achieve broader catalog exposure. More support improves ItemCF on the common cohort, but Content's NDCG is not monotonic. These outcomes demonstrate why the study reports multiple metrics and retains unfavorable results rather than equating personalization with improvement.

The work is complete at the defined midterm technical scope, not at the final research scope. Future conclusions about BPR, hybrid scoring, diversity-oriented reranking, statistical robustness, or real-world user benefit require additional experiments. Preserving the present artifacts and protocol provides a reliable baseline against which those extensions can be assessed.

References

[1] J. McAuley, “Recommender Systems Datasets,” UC San Diego, Steam dataset collection. Available: https://cseweb.ucsd.edu/~jmcauley/datasets.html. Accessed: Sep. 25, 2026. Exact downloaded files and hashes are recorded in the project's data manifests.

[2] scikit-learn developers, “Feature extraction: TF–IDF term weighting,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/feature_extraction.html. Accessed: Sep. 25, 2026. The project implementation uses pinned scikit-learn 1.6.1; the web documentation is a conceptual reference.

[3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in Proceedings of the 10th International Conference on World Wide Web, 2001, pp. 285–295. doi: 10.1145/371920.372071. Author paper: https://www.ra.ethz.ch/CDstore/www10/papers/519/.

[4] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002. doi: 10.1145/582415.582418.

[5] H. Abdollahpouri, M. Mansoury, R. Burke, and B. Mobasher, “The Unfairness of Popularity Bias in Recommendation,” arXiv:1907.13286, 2019. Available: https://arxiv.org/abs/1907.13286.

[6] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, “BPR: Bayesian Personalized Ranking from Implicit Feedback,” in Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence, 2009, pp. 452–461. Available: https://arxiv.org/abs/1205.2618.

[7] Playnext project team, “Frozen M1 experiment and M3 benchmark,” local reproducibility artifacts, September 2026. M1 run: 20260915T050327853533Z-seed42. M3 benchmark: 20260918T065012656328Z. Includes metrics, cohort audits, configuration selection, predictions, manifests, and raw timing samples.

Appendix A Reproduction and Evidence Map

The raw-data survey is implemented in scripts/survey_data.py. User splitting and case construction are in ml/data.py; the three baselines and their persistence interface are in ml/models.py. Ranking metrics and aggregate evaluation are in ml/metrics.py and ml/evaluation.py. The pipeline entry point is ml/run.py, configured by configs/m1.json and configs/m1-data.json. The online adapter is backend/app/recommender.py.

To recreate a research run, create a Python 3.11 environment, install requirements.lock, execute scripts/download_data.ps1, and run python -m ml.run from the project root. Run python -m scripts.verify_m1 to audit the latest generated run. Each execution creates a new run identifier. Do not rename a new experiment to the historical midterm identifier or overwrite the saved midterm snapshot.

Experiment reports are stored under experiments/m1/<run_id>. Model arrays, split manifests, cases, and per-user predictions are under artifacts/m1/<run_id>. The latter and the raw data are excluded from the source-only handoff. For the website, follow BAN_GIAO.md, install backend/requirements.lock, configure MODEL_RUN_ID to the available run, and start the local PostgreSQL/API application using the documented script.

The report's charts are generated from frozen metrics, data summaries, popularity arrays, and benchmark timing records. The worked formula examples are explicitly hypothetical. The selected failure case is derived from existing development-test predictions and is used only for explanation; no model is retrained or tuned from that case. This separation allows another reader to distinguish measured claims, explanatory examples, and proposed work.

Appendix B Team Contributions and Responsibilities

Group 6 assigns the following responsibilities for the midterm submission and presentation. The allocation covers technical explanation, report review, reproducibility checks and demonstration preparation. It records assigned responsibilities rather than verified completion or historical contribution percentages.

Team member	Assigned responsibilities
Nguyễn Bá Hùng 523K0006	Data provenance, positive-label definition, cleaning, metadata joins and exploratory analysis. Explanation of Most Popular, TF-IDF/cosine similarity and ItemCF, including worked examples. Review of Chapters 1–4 and the algorithm sections of the IEEE report; verification of formulas and references. Final package and checksum review; presentation of the problem, data and baseline algorithms.
Nguyễn Bảo Long 523K0014	Explanation of user-disjoint splitting, support/query construction, Recall, NDCG, coverage and diversity. Interpretation of baseline results, negative findings and study limitations. Review of Chapters 5–9 and the appendices, report formatting and the evaluation sections of the IEEE report. Second-machine setup and reproducibility checks; preparation of the website demonstration and presentation of protocol and results.

Shared responsibilities

Both members cross-check the other member’s technical explanations, preserve the frozen midterm metrics, rehearse the complete demonstration and prepare to answer questions about all three baselines. Both review the final documents and confirm the lecturer’s submission requirements.

Completion status

Second-machine installation, confirmation of the course-specific report template and presentation rehearsal remain to be confirmed by the members. The allocation does not attribute previously completed implementation work or certify equal contribution percentages.